The Kryptos Benchmark: Evaluating Large Language Models on Classical Cryptanalysis and Character-Level Reasoning

The intersection of artificial intelligence and cryptanalysis represents a highly critical frontier for measuring advanced mathematical reasoning, algorithmic execution, and cybersecurity capabilities. Historically, the evaluation of Large Language Models (LLMs) within cryptographic domains has been largely relegated to fundamental substitution ciphers or artificially simplified capture-the-flag problems. However, the introduction of rigorous frameworks in 2026, such as CryptanalysisBench, has conclusively demonstrated that frontier models possess the latent analytical capacity to discover novel attacks against production-grade cryptographic primitives, including scaled-down AES and post-quantum candidates like HAWK[1].

To systematically quantify the capacity of LLMs to perform complex, multi-stage classical cryptanalysis, a benchmark modeled after the world's most prominent unsolved cryptographic puzzle—the Kryptos sculpture at Central Intelligence Agency Headquarters—provides an optimal evaluation matrix. Created by artist Jim Sanborn in collaboration with former CIA Cryptographic Center Chairman Edward Scheidt in 1990, Kryptos contains four distinct ciphertext passages (K1 through K4) of escalating complexity[2]. While the first three passages were deciphered by human analysts in the late 1990s, the K4 passage remains an enduring mathematical anomaly[2].

Constructing a standardized "Kryptos Benchmark" for LLMs introduces a profound methodological challenge: Benchmark Data Contamination (BDC). Because the Kryptos ciphertexts and their corresponding solutions have been extensively documented across academic papers, cryptographic forums, and encyclopedic databases, frontier LLMs have almost certainly ingested these plaintext-ciphertext pairs during their pre-training phases. Consequently, testing models on the exact strings physically carved into the sculpture would merely evaluate the model's capacity for probabilistic memory retrieval rather than its capacity for algorithmic cryptanalysis.

This report provides an exhaustive architectural blueprint for engineering a contamination-resistant Kryptos Benchmark. It delineates the cryptographic mathematics required to synthesize dynamic evaluation datasets, addresses the subword tokenization bottlenecks that inherently restrict character-level reasoning in transformer architectures, and outlines a rigorous evaluation methodology utilizing code-execution environments and Levenshtein-based error metrics.

The Cryptographic Architecture of the Kryptos

Passages

To synthesize a valid and comprehensive benchmark, the underlying encryption mechanisms of the historical Kryptos passages must be reverse-engineered and parametrically abstracted. The physical installation comprises 1,735 alphabetic letters cut into a verdigrised copper screen, structurally divided into a left panel containing the ciphertexts and a right panel presenting a Vigenère tableau[2]. The cryptographic complexity escalates systematically through the installation, integrating environmental clues ranging from lodestones and compass roses to Morse code panels inscribed with phrases such as VIRTUALLY INVISIBLE and LUCID MEMORY[2].

Passage	Cryptographic Primitive	Cryptanalytic Complexity	Key Constraints & Anomalies
K1	Quagmire III (Polyalphabetic)	Period determination, frequency analysis	Keyword PALIMPSEST, intentional typo IQLUSION
K2	Quagmire III (Polyalphabetic)	Period determination, symmetry extraction	Keyword ABSCISSA, intentional typo UNDERGRUUND
K3	Double Columnar Transposition	Geometric reshaping, anagramming	Matrix rotations, intentional typo DESPARATLY
K4	Unsolved (Masking Hypothesis)	Multi-layered or asymmetric algebra	Extra 'L' in tableau, 'W' segment separators

Polyalphabetic Substitution: The Quagmire III Matrices

The first two passages of Kryptos are encrypted utilizing a Quagmire III cipher, a sophisticated polyalphabetic substitution system that relies on two distinct keywords to generate both the substitution alphabet and the cyclical shift period. Unlike a standard Vigenère cipher, which utilizes a standard alphabetical sequence, Quagmire III utilizes a customized, keyed alphabet for both the plaintext axis and the ciphertext axis.

In the Kryptos implementation, the primary keyword used to generate the mixed alphabet is KRYPTOS. Extraneous duplicate letters are mathematically filtered, and the remaining letters of the Latin alphabet are appended sequentially, resulting in the foundational mixed alphabet

sequence: KRYPTOSABCDEFGHIJLMNQUVWXZ. The second keyword acts as the indicator, determining the operational period and the specific shift alignments of the ciphertext alphabets across the tableau. For K1, the indicator keyword is PALIMPSEST, which establishes a period of 10. For K2, the keyword is ABSCISSA, establishing a period of 8[2].
For an LLM to successfully cryptanalyze a Quagmire III cipher without prior knowledge of the keys, it must execute a highly deterministic, multi-step mathematical procedure. First, the model must compute the Index of Coincidence (IoC) to deduce the underlying period. The IoC measures the statistical probability of drawing two matching letters randomly from a given text. The formal mathematical formulation is defined as:

$$IoC = \frac{\sum_{i=A}^{Z} f_i(f_i - 1)}{N(N-1)}$$

In this equation, f_i represents the frequency of the i-th letter in the sequence, and N represents the total character length of the text. The algorithmic period is identified by slicing the ciphertext into l columns and finding the integer l that yields an aggregate IoC approximating the standard English linguistic average of 0.066. Once the period is established, the LLM must utilize advanced frequency analysis across the isolated columns, leveraging the inherent symmetries of the Quagmire III matrix. Because the alphabet is identical across all rows, the numerical distance between any two specific letters in the same row or column remains a constant integer across all shifting alphabets, enabling the reconstruction of the primary keyword through localized hill-climbing optimization.

Geometric Transposition: Double Columnar Routing

The third passage introduces a severe structural departure from substitution, utilizing a complex route transposition technique operating on a predefined width of 86. Rather than substituting letters for alternative characters, a transposition cipher geometrically permutes the physical positions of the characters while preserving the underlying monogram frequencies of the original plaintext.
Cryptanalytic reconstruction of the K3 solution reveals a multi-stage geometric transformation matrix. The plaintext is padded, written into a rectangular grid, and systematically extracted via a specific directional path. The exact mathematical sequence involves padding the text and writing it sequentially into a 24×14 matrix. The entire matrix subsequently undergoes a 90° clockwise rotation. Following this rotation, the rows are sliced into uniform segments of 8 characters to form a vertically elongated 42×8 matrix. A final 90° clockwise rotation produces an 8×42 matrix, from which the ultimate ciphertext is extracted by reading downwards through the columns.
For an LLM to algorithmically defeat a complex transposition cipher, statistical frequency analysis is entirely useless, as the IoC remains mathematically identical to standard English.

Instead, the model must deploy sophisticated anagramming algorithms, multi-stage recursive matrix reshaping, or combinatorial hill-climbing algorithms that assess the linguistic fitness of various column permutations using n-gram scoring models.

The K4 Frontier and Masking Hypotheses

The 97-character K4 passage remains the most critical unresolved puzzle in modern classical cryptography. Artist Jim Sanborn has released four specific plaintext positional cribs to the public: BERLIN at positions 64–69, CLOCK at positions 70–74, NORTHEAST at positions 26–34, and EAST at positions 22–25[2]. Despite providing 24 characters of the 97-character plaintext, the underlying generative mechanism has completely eluded traditional cryptanalysis. While a highly publicized 2025 archival discovery by researchers Jarett Kobek and Richard Byrne recovered fragmented plaintext sentences indicating a connection to a hypothetical "K5" continuation, the researchers explicitly confirmed they did not recover the cryptographic method, the key, or the coding charts, leaving the cipher structurally unbroken.

Edward Scheidt indicated that the method involves an asymmetric "masking technique" that requires a fundamental conceptual paradigm shift rather than a mere escalation in mathematical complexity[2]. Several highly developed hypotheses exist regarding the internal mechanics of K4, which must be incorporated into the benchmark's advanced evaluation tiers to test an LLM's capacity for hypothesis generation.

The Hill Cipher conjecture, formalized by researchers Bauer, Link, and Molle, posits that an anomalous extra 'L' deliberately carved into the sculpture's Vigenère tableau results in the consecutive vertical string H-I-L-L[2]. A Hill cipher relies strictly on linear algebra, specifically modulo 26 matrix multiplication. A block of plaintext is converted to a column vector P and multiplied by an $n \times n$ key matrix K, expressed as $C \equiv K \cdot P \pmod{26}$. An LLM attempting to solve this would need to programmatically construct an adjugate matrix to calculate the inverse K^{-1} and execute modulo 26 arithmetic over a known-plaintext attack utilizing the BERLIN CLOCK cribs[3].

Alternatively, the segment separator hypothesis relies on statistical anomaly detection. The character 'W' appears exactly 6 times within the 97 characters of K4 and may act as a null delimiter, cleanly separating the ciphertext into six functional segments that exhibit an alternating $A|B|A|B|A|B$ letter frequency distribution, implying a highly customized bifid or dual-encryption routing mechanism[4]. Testing LLMs against these hypotheses evaluates their capacity to synthesize environmental clues into actionable cryptographic code.

The Threat of Benchmark Data Contamination

When evaluating advanced neural architectures on the Kryptos puzzles, the primary empirical threat is Benchmark Data Contamination (BDC). Due to the aggressive web-scraping methodologies utilized to assemble LLM pre-training corpora, frontier models have inevitably ingested encyclopedic articles, GitHub repositories, cryptographic forums, and academic

papers detailing the exact mathematical solutions to K1, K2, and K3.
If a static benchmark simply prompts an LLM to decode the string EMUFPHZLRFA... (the beginning of K1), the model will almost certainly output the correct plaintext (BETWEEN SUBTLE SHADING...) via probabilistic memory retrieval rather than applying genuine cryptanalytic deduction. This phenomenon generates falsely inflated performance estimates, rendering the evaluation epistemologically void.

Mitigation Strategy	**Mechanism**	**Vulnerability in Cryptanalysis Context**
N-Gram Overlap Filtering	Removes test strings found in training data.	Destroys the domain-specific evaluation entirely by removing the target ciphers.
Perplexity Measurement	Flags sequences with unnaturally low perplexity.	Fails to verify if the model can actually execute the underlying decryption algorithms.
Dynamic Isomorphic Synthesis	Generates novel ciphertexts using identical historical algorithms.	**Optimal.** Isolates algorithmic reasoning from static data memorization.

Traditional BDC mitigation strategies primarily rely on n-gram overlap detection to permanently filter contaminated samples from the evaluation set, or perplexity-based filtering to assess if a model has memorized a specific string. However, these reactive strategies are entirely insufficient for highly specific cryptanalysis tasks. The core objective of the benchmark is to test the specific cryptographic algorithms utilized in the Kryptos installation. If the Kryptos ciphertexts are removed entirely, the domain-specific evaluation is lost. Furthermore, contemporary research on dataset contamination clearly indicates that no existing static mitigation strategy effectively balances evaluation fidelity with robust contamination resistance. To preserve the utility of the benchmark while completely neutralizing memorization artifacts, a dynamic, generative evaluation architecture must be adopted.

Architecting the Dynamic Dataset Generation Pipeline

To measure genuine cryptanalytic capability, the benchmark must dynamically synthesize "Kryptos-Isomorphic" datasets at inference time. An isomorphic cipher applies the exact mathematical operations, matrix transformations, and algorithmic constraints utilized in the historical Kryptos passages, but applies them to entirely novel, unseen plaintexts utilizing novel, randomized keywords. By generating synthetic data programmatically, the benchmark

guarantees a zero-percent probability of pre-training contamination while maintaining 100% structural fidelity to the challenges designed by Jim Sanborn and Ed Scheidt.

The generation pipeline requires a robust, custom Python execution environment. Leveraging open-source synthetic data generator frameworks, combined with algorithmic implementations of classical ciphers, serves as the computational engine for this benchmark.

The initial phase requires rigorous plaintext curation. The source material must consist of diverse text fragments guaranteed to be unseen by the model prior to the evaluation. This is achieved by utilizing secondary LLMs to generate highly specific synthetic text—such as short narratives detailing fictional geopolitical events or obscure technical manuals—or by extracting text from localized, non-public databases immediately prior to execution. The diversity of the text domains prevents the evaluation model from relying on semantic format biases. Once generated, the texts must be strictly normalized: stripped of all punctuation, converted to uppercase, and stripped of spacing to mathematically mirror the continuous string format of the original copper sculpture.

Following normalization, the isomorphic encryption pipelines are engaged. For K1 and K2 isomorphs, the pipeline programmatically selects a randomized primary alphabet keyword (e.g., CIPHER) and a randomized indicator keyword (e.g., ENIGMA). A custom Python script dynamically generates the corresponding 26×26 keyed substitution matrix, filters duplicate characters, and shifts the tableau according to the indicator. The synthetic plaintext is subsequently encrypted through this matrix, generating a cryptographically identical challenge to Quagmire III that is entirely insulated from data leakage.

For K3 isomorphs, the pipeline instantiates a rectangular matrix defined by randomized $M \times N$ dimensions. The synthetic text is written into the grid, rotated by specific degrees, sliced into sub-matrices, and extracted via complex columnar read-out paths. To simulate the unknown conditions of the K4 passage, the pipeline generates composite ciphers. This includes applying a Vigenère substitution followed immediately by a Hill cipher matrix multiplication, or applying a Quagmire cipher interspersed with null separators based on the 'W' hypothesis[3]. These multi-layered synthetic ciphers provide a highly accurate mathematical proxy for testing hypothesis generation on complex masking techniques.

The Tokenization Bottleneck and Character-Level Reasoning

A severe, often overlooked structural limitation dictates exactly how LLMs interact with cryptographic data: subword tokenization. Frontier LLMs do not process text as individual, discrete characters. Instead, they process text as subword tokens via compression algorithms such as Byte-Pair Encoding (BPE) or WordPiece.

BPE optimizes strictly for grammatical text compression, progressively merging frequent character pairs found in the training corpus until a predefined vocabulary size is reached. For standard English prose, BPE achieves an efficient compression ratio of 3 to 5 characters per token. However, cryptography operates strictly on an absolute character-by-character basis.

When a continuous, unspaced ciphertext string like EMUFPHZLRFA is passed to an LLM, the tokenizer does not recognize 11 discrete letters. It parses the string into arbitrary subword tokens based on historical frequencies, potentially viewing it as [EMU] [FPHZL] [RFA].
This subword abstraction entirely obscures the specific character-level positional indices, preventing the LLM's attention mechanism from applying a discrete matrix shift to the 6th character of the string. Research on character-level tasks, such as those conducted via CharBench, highlights that model accuracy declines significantly as the target token length increases, demonstrating a stark tradeoff between token-level compression efficiency and the model's fundamental ability to reason about internal character structures. Furthermore, modern tokenizers routinely produce non-unique encodings where multiple token ID sequences detokenize to identical surface strings, establishing a representational mismatch that systematically betrays mathematical reasoning and induces "phantom edits".

Tokenization Variable	**Impact on Cryptanalysis**	**Benchmark Mitigation Strategy**
Subword Merging (BPE)	Obscures character indices, preventing accurate modulo arithmetic on individual letters.	Force character delimitation (insert spaces between all ciphertext letters).
Non-Unique Encodings	Causes phantom edits; models believe they have altered text when surface strings remain identical.	Evaluate via external code execution rather than internal autoregressive state logic.
Format Constraints	Strict JSON schemas can alter the token distribution, degrading underlying reasoning accuracy.	Provide few-shot format demonstrations to stabilize probability distributions.

To bypass this critical tokenization bottleneck, the Kryptos Benchmark must employ highly specific prompt formatting techniques. Recent empirical studies on format bias reveal that the structural presentation of a prompt—whether formatted as plain text, Markdown, JSON, or YAML—can alter an LLM's performance variance by up to 40%.
To optimize cryptanalytic reasoning, the benchmark applies character delimitation transformations. The generated ciphertext string must be pre-processed to include a space or unique delimiter between every individual character (e.g., E M U F P H Z L R F A). This intentional spacing forces the BPE tokenizer to assign a unique token ID to every individual letter, perfectly aligning the model's internal representation with the discrete character-level logic strictly required for substitution algorithms. Additionally, prompting the LLM to output its reasoning

inside a strict JSON schema allows for automated evaluation, but requires few-shot demonstrations to prevent the strict grammar constraints from degrading the model's underlying reasoning performance.

Evaluation Paradigms: Autoregressive CoT vs. Programmatic Tool-Use

The evaluation of the LLM's decryption capabilities must be tested across two distinct operational paradigms: purely autoregressive Chain-of-Thought (CoT) prompting and programmatic Tool-Use environments utilizing code execution.

Zero-shot and Few-shot CoT prompting compel the LLM to process the cipher sequentially within its internal neural weights. While zero-shot CoT techniques (such as appending "Let's think step by step") significantly improve semantic logical deduction, they frequently fail catastrophically on high-complexity deterministic mathematics. For a polyalphabetic cipher with a period of 10, the LLM must independently track 10 different shifting alphabets simultaneously and apply the correct modulo 26 addition to each subsequent character without error.

Given the architectural lack of a genuine internal scratchpad, LLMs frequently hallucinate intermediate mathematical steps. Theoretical research on Execution Trace Chain-of-Thought (ET-CoT) indicates that autoregressive transformers struggle profoundly to emulate a Turing machine without highly explicit, granular execution traces. Attempting to force an LLM to perform computations that exceed its constant-depth (TC^0) processing limitations via standard CoT inevitably leads to cascading arithmetic errors.

Because cryptanalysis provides exact formal specifications and allows for automatic, zero-variance verification, it is an ideal candidate for programmatic reasoning[1]. Rather than attempting to "guess" the decryption via next-token prediction, the LLM should be evaluated strictly on its ability to write, debug, and execute Python code to perform the cryptanalysis. The benchmark integrates a sandboxed Code Execution environment. The LLM is prompted to act as an autonomous cryptographic agent. It must analyze the ciphertext, formulate a hypothesis, write a Python script using standard libraries or specialized tools, execute the script, and evaluate the output.

This methodology mirrors modern automated cryptanalysis tools, such as the decipher suite, which layers LLM investigation loops over fast, Rust-accelerated simulated annealing engines and n-gram frequency scoring. By delegating the rote mathematics to a Python interpreter, the benchmark decouples the advanced semantic reasoning capabilities of the LLM from its inherent arithmetic limitations.

Quantitative Metrics: CER and Levenshtein Distance

Measuring the success of an LLM in a cryptanalysis benchmark requires highly specialized metrics that tolerate partial successes and side-channel artifacts. Standard natural language generation metrics like BLEU, ROUGE, or simple Exact Match (EM) are fundamentally inappropriate for cryptography. If an LLM's decryption is entirely correct but shifts by a single

letter due to an off-by-one error in a matrix rotation, a rigid Exact Match metric will score it as a 0% success, masking the model's actual algorithmic progress.

The primary evaluation metric for the Kryptos Benchmark is the Character Error Rate (CER), anchored by the Levenshtein distance algorithm. The Levenshtein distance calculates the absolute minimum number of single-character edits required to change the model's generated output into the correct ground-truth reference plaintext.

The mathematical formulation for CER is calculated as:

$$CER = \frac{S + D + I}{N} \times 100$$

Where S represents the number of character substitutions, D represents the number of character deletions, I represents the number of character insertions, and N represents the total number of characters in the ground-truth reference text.

Metric	Evaluation Focus	Cryptanalytic Utility	Limitation
Exact Match (EM)	Binary pass/fail	Low. Fails to capture near-miss algorithmic successes.	Intolerant of f or minor arith errors.
Word Error Rate (WER)	Lexical word accuracy	Moderate. Useful for semantic checks.	Fails on contir unspaced stri typical in cryp
Character Error Rate (CER)	Lexical character edits	**High.** Captures exact positional accuracy.	Semantic blin treats all error regardless of

CER provides granular visibility into the lexical accuracy of the decryption. In cryptography, a purely semantic metric is blind to critical failures; misinterpreting the direction "WEST" as "EAST" has catastrophic implications in a coordinate cipher, despite their close semantic similarity in

natural language processing. A lower CER indicates higher accuracy, with 0% representing a perfect cryptanalytic break. For highly complex, multi-layered ciphers, a CER below 15% often indicates that the core cryptographic algorithm was successfully broken, with the remaining errors attributed to minor misalignments or the presence of intentional spelling anomalies, such as Sanborn's deliberate inclusions of IQLUSION or DESPARATLY[2].

To execute these mathematical evaluations efficiently at scale, the benchmark utilizes rapidfuzz, a high-performance string matching library implemented in C++ with Python bindings. The rapidfuzz library vastly outperforms pure Python implementations of Levenshtein metrics by leveraging optimized CPU features, short-circuiting algorithms, and avoiding external compiled dependencies. The evaluation script utilizes the normalized ratio function, which calculates the normalized Levenshtein distance on a scale from 0 to 100. This normalization mathematically ensures that the metric scales appropriately across passages of vastly varying lengths, ranging from the dense 97-character K4 segment to the expansive 869-character left panel of K1 and K2.

The Kryptos Benchmark Framework

Integrating the isomorphic data generation pipeline, tokenization formatting mitigations, programmatic execution environments, and CER evaluation metrics, the final Kryptos Benchmark is structured across four distinct tiers of increasing cryptanalytic difficulty. This tiered architecture prevents a binary pass/fail assessment and allows for a highly nuanced, quantifiable assessment of an LLM's cryptanalytic maturity.

Tier	Designation	Cryptographic Input	Evaluated Capability	Success Threshold
1	Algorithmic Identification	Synthetic ciphertext, cipher name, exact keys	Python code execution, basic implementation	0% CER
2	Single-Layer Cryptanalysis	Synthetic Quagmire III ciphertext (No keys)	IoC calculation, frequency analysis, hill-climbing	CER < 5%
3	Geometric Transposition	Synthetic Double Columnar ciphertext	Spatial reasoning, n-gram optimization, anagramming	CER < 10%

4	The K4 Frontier	Authentic K4 ciphertext + BERLIN CLOCK cribs	Hypothesis generation, matrix algebra, masking analysis	Normalized Levenshtein > 30%

The first tier establishes a baseline capability, testing whether the LLM can simply follow instructions to implement a known algorithm using provided keys. The second tier evaluates automated cryptanalysis, requiring the LLM to autonomously execute Index of Coincidence analysis, determine the period, and utilize dictionary-based hill-climbing to recover the primary and indicator keywords. The third tier pushes into spatial reasoning, requiring the model to recognize the preservation of monogram frequencies, classify the text as a transposition, and write recursive algorithms to test varying matrix dimensions.
The fourth and final tier represents the absolute frontier of AI cryptanalysis. The model is provided with the authentic 97-character K4 ciphertext alongside Sanborn's plaintext cribs and architectural anomalies. The model must generate and execute novel cryptanalytic attack vectors, such as constructing complex modulo 26 Hill cipher solvers or multi-stage masking arrays. While a full solution to K4 is not expected given its historical resilience, the benchmark measures the LLM's computational ability to formulate mathematically sound hypotheses that align with the known plaintext fragments, pushing the boundaries of autonomous algorithmic discovery.

Works cited

1. CryptanalysisBench: Can LLMs do Cryptanalysis? - arXiv, https://arxiv.org/html/2607.18538v1
2. Kryptos - The Cipher Museum, https://ciphermuseum.com/ciphers/kryptos.html
3. Kryptos - The Cipher (Part 2) - K4 abnormalities and solution attempts, https://www.numberworld.blog/2017/03/kryptos-cipher-part-2.html
4. A Fresh Perspective on Kryptos K4, the Decades-Old Unsolved Code, https://glthr.com/a-fresh-perspective-on-kryptos-k4